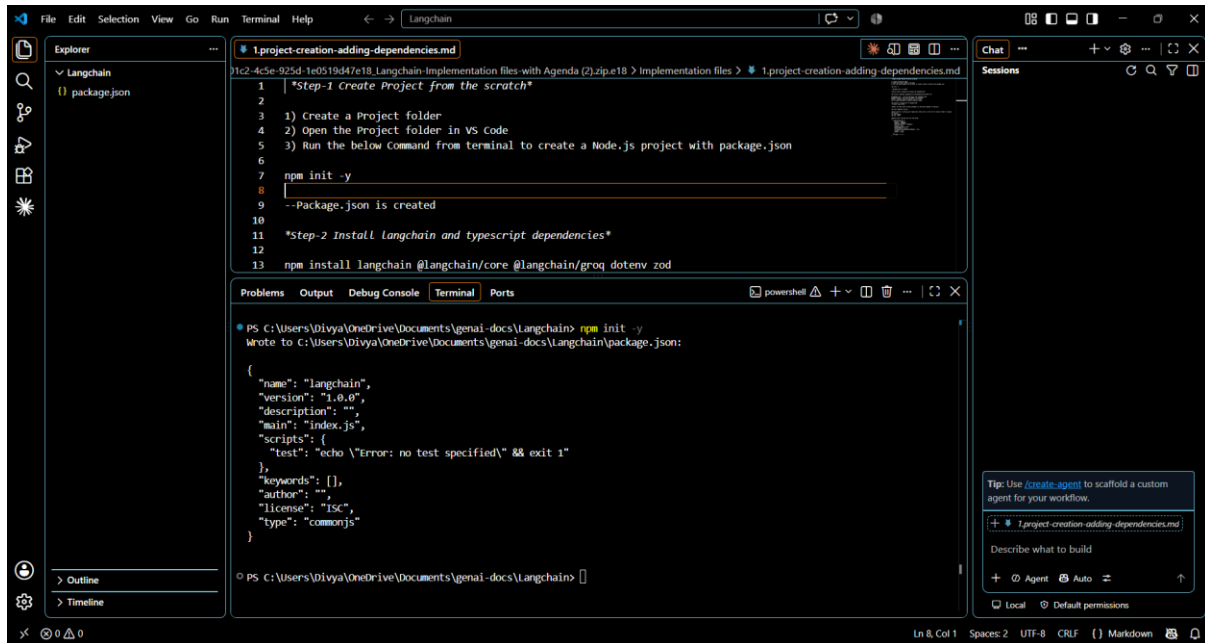


Step-1 Create Project from the scratch

- 1) Create a Project folder
- 2) Open the Project folder in VS Code
- 3) Run the below Command from terminal to create a Node.js project with package.json

`npm init -y`

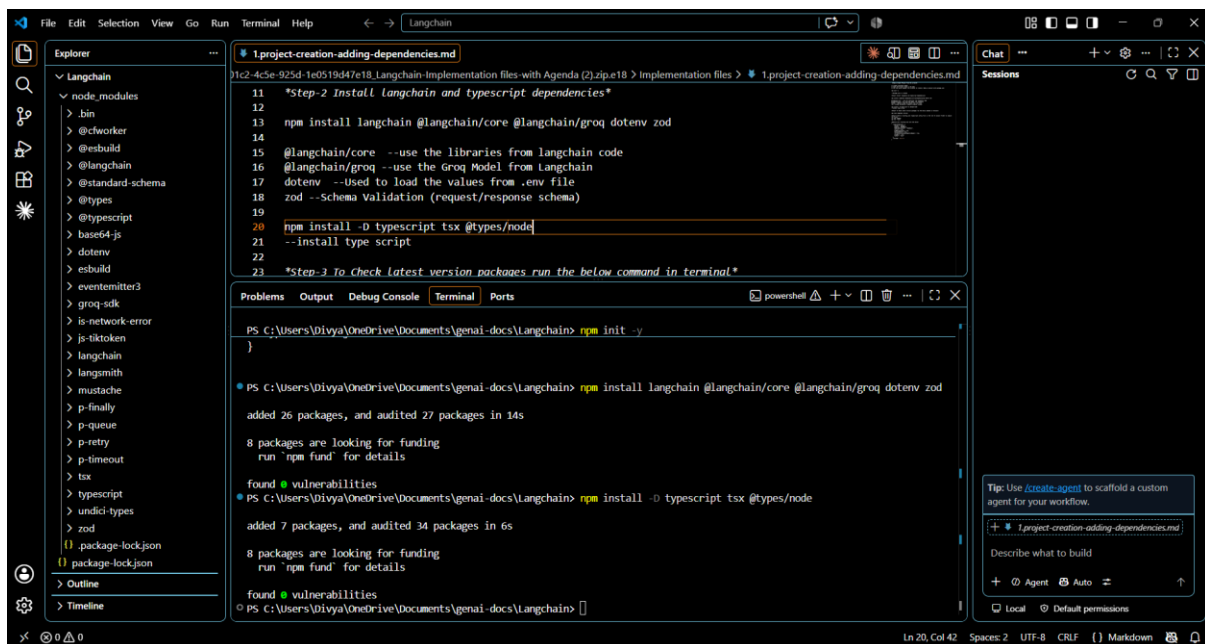
--Package.json is created



Step-2 Install langchain and typescript dependencies

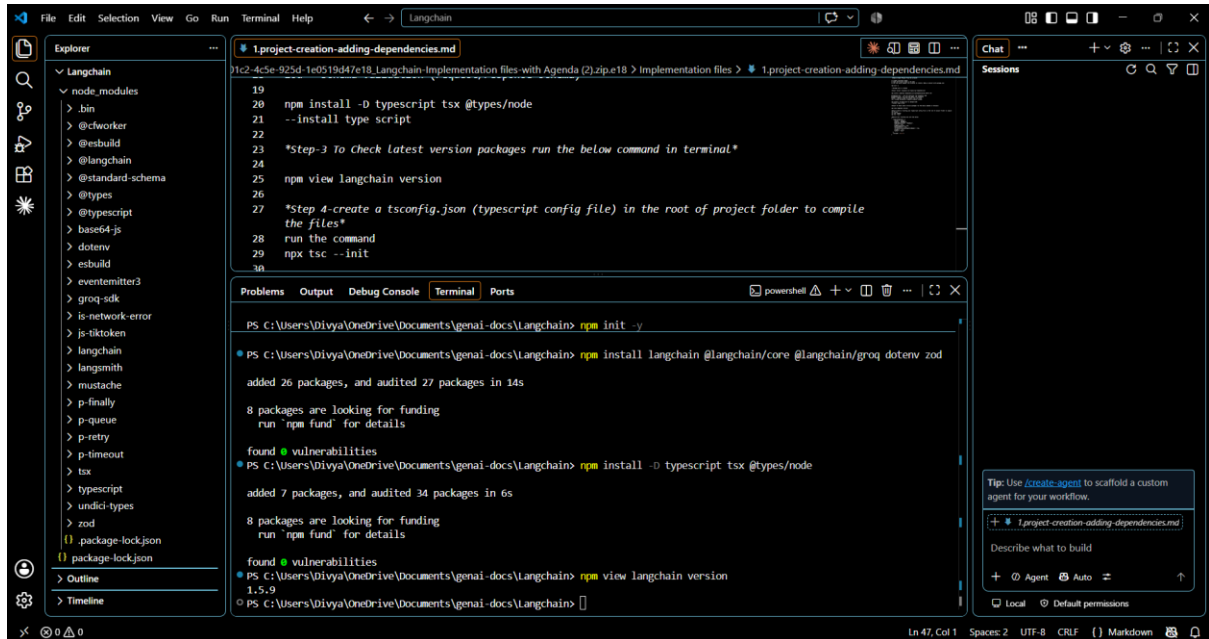
`npm install langchain @langchain/core @langchain/groq dotenv zod`

`npm install -D typescript tsx @types/node`



Step-3 To Check latest version packages run the below command in terminal

npm view langchain version



```
19
20 npm install -D typescript tsx @types/node
21 --install type script
22
23 *Step-3 To Check latest version packages run the below command in terminal.*
24
25 npm view langchain version
26
27 *Step 4-create a tsconfig.json (typescript config file) in the root of project folder to compile
the files*
28 run the command
29 npx tsc --init
30
```

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm init -y

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm install langchain @langchain/core @langchain/groq dotenv zod

added 26 packages, and audited 27 packages in 14s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm install -D typescript tsx @types/node

added 7 packages, and audited 34 packages in 6s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm view langchain version

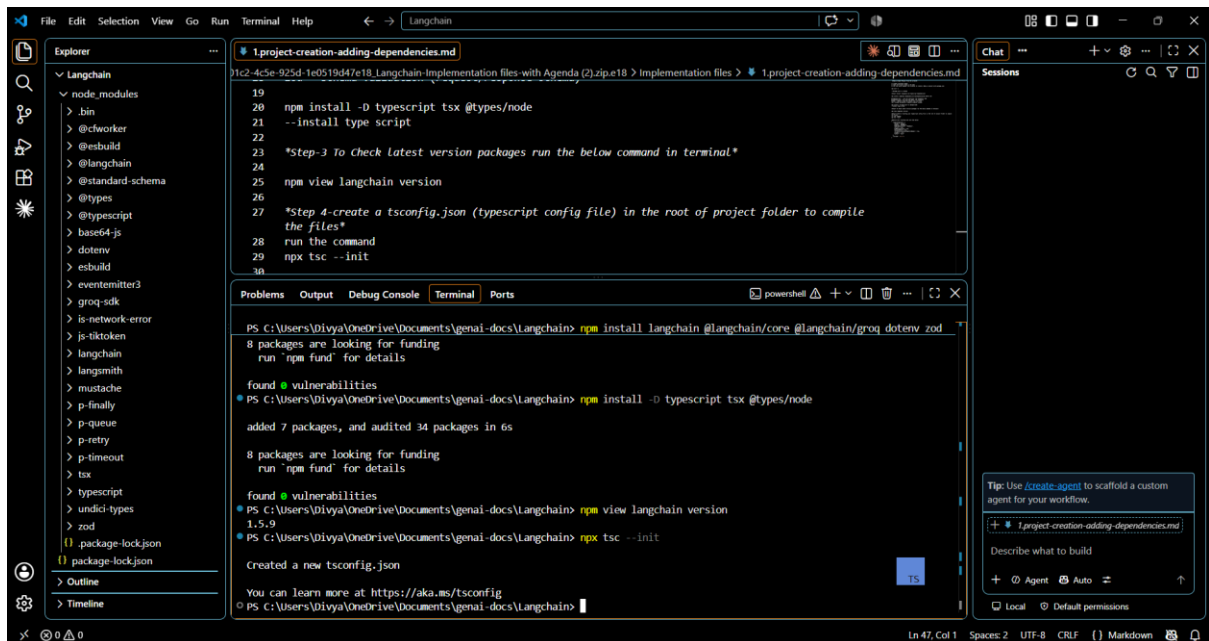
1.5.9

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>

Step 4-create a tsconfig.json (typescript config file) in the root of project folder to compile the files

run the command

npx tsc --init



```
19
20 npm install -D typescript tsx @types/node
21 --install type script
22
23 *Step-3 To Check latest version packages run the below command in terminal.*
24
25 npm view langchain version
26
27 *Step 4-create a tsconfig.json (typescript config file) in the root of project folder to compile
the files*
28 run the command
29 npx tsc --init
30
```

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm install langchain @langchain/core @langchain/groq dotenv zod

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm install -D typescript tsx @types/node

added 7 packages, and audited 34 packages in 6s

8 packages are looking for funding
run 'npm fund' for details

found 0 vulnerabilities

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npm view langchain version

1.5.9

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> npx tsc --init

Created a new tsconfig.json

You can learn more at <https://aka.ms/tsconfig>

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>

Replace your tsconfig.json with the below

```
{

"compilerOptions": {

"target": "ES2022",

"module": "NodeNext",
```

```

"moduleResolution": "NodeNext",

"strict": true,

"esModuleInterop": true,

"skipLibCheck": true,

"forceConsistentCasingInFileNames": true,

"rootDir": "src",

"outDir": "dist"

},

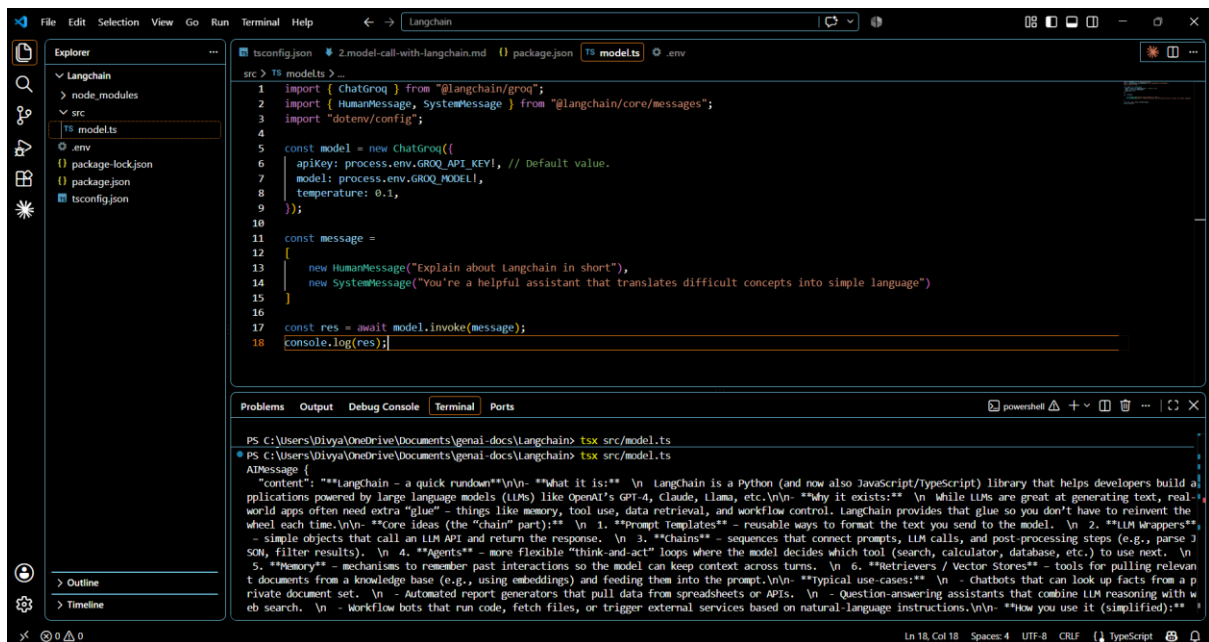
"include": ["src"]

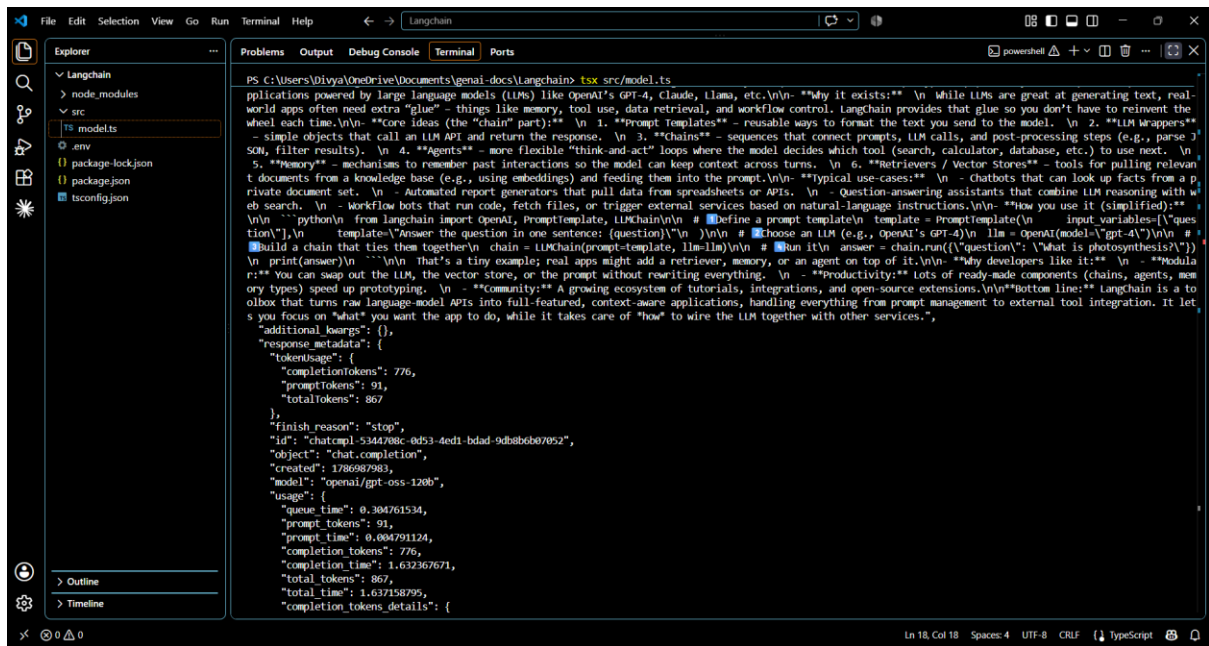
}

```



Model.ts





The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the project structure: `Langchain`, `node_modules`, `src`, `ts models.ts`, `.env`, `package-lock.json`, `package.json`, and `tsconfig.json`. The main editor area shows the `ts models.ts` file with the following content:

```
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/models.ts

Applications powered by large language models (LLMs) like OpenAI's GPT-4, Claude, Llama, etc.

Why it exists: While LLMs are great at generating text, real-world apps often need extra "glue" - things like memory, tool use, data retrieval, and workflow control. LangChain provides that glue so you don't have to reinvent the wheel each time.

Core ideas (the "chain" part):
1. Prompt Templates - reusable ways to format the text you send to the model.
2. LLM Wrappers - simple objects that call an LLM API and return the response.
3. Chains - sequences that connect prompts, LLM calls, and post-processing steps (e.g., parse JSON, filter results).
4. Agents - more flexible "think-and-act" loops where the model decides which tool (search, calculator, database, etc.) to use next.
5. Memory - mechanisms to remember past interactions so the model can keep context across turns.
6. Retrievers / Vector Stores - tools for pulling relevant documents from a knowledge base (e.g., using embeddings) and feeding them into the prompt.

Typical use-cases:
- Chatbots that can look up facts from a private document set.
- Automated report generators that pull data from spreadsheets or APIs.
- Question-answering assistants that combine LLM reasoning with web search.
- Workflow bots that run code, fetch files, or trigger external services based on natural-language instructions.

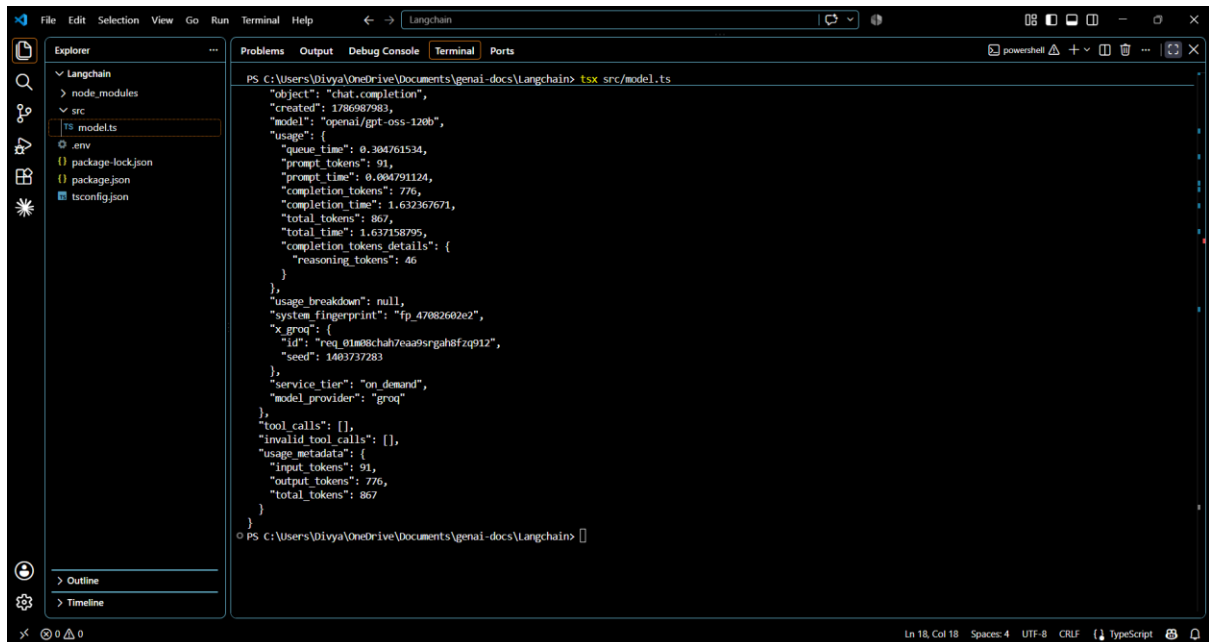
How you use it (simplified):
python
from langchain import OpenAI, PromptTemplate, LLMChain
# Define a prompt template
template = PromptTemplate(
    input_variables=["question"],
    output_parser=OpenAIParser()
)
# Choose an LLM (e.g., OpenAI's GPT-4)
llm = OpenAI(model="gpt-4")
# Build a chain that ties them together
chain = LLMChain(prompt_template, llm)
# Run it
answer = chain.run("What is photosynthesis?")
print(answer)

That's a tiny example; real apps might add a retriever, memory, or an agent on top of it.

Why developers like it:
- Modularity: You can swap out the LLM, the vector store, or the prompt without rewriting everything.
- Productivity: Lots of ready-made components (chains, agents, new or types) speed up prototyping.
- Community: A growing ecosystem of tutorials, integrations, and open-source extensions.

Bottom line: LangChain is a toolbox that turns raw language-model APIs into full-featured, context-aware applications, handling everything from prompt management to external tool integration. It lets you focus on "what" you want the app to do, while it takes care of "how" to wire the LLM together with other services.

Additional keys: {},
Response metadata: {
  "tokenUsage": {
    "completionTokens": 776,
    "promptTokens": 91,
    "totalTokens": 867
  },
  "finishReason": "stop",
  "id": "chatcmpl-5344208c-0d53-4ed1-bdad-9db8b6b07052",
  "object": "chat.completion",
  "created": 1786987983,
  "model": "openai/gpt-oss-120b",
  "usage": {
    "queue_time": 0.304761534,
    "prompt_tokens": 91,
    "prompt_time": 0.004791124,
    "completion_tokens": 776,
    "completion_time": 1.632367671,
    "total_tokens": 867,
    "total_time": 1.637158795,
    "completion_tokens_details": {
      "reasoning_tokens": 46
    }
  },
  "usageBreakdown": null,
  "system_fingerprint": "fp_4708260202",
  "x_groq": {
    "id": "req_01m08chah7eaa9srgah8fzq912",
    "seed": 1403737283
  },
  "serviceTier": "on_demand",
  "model_provider": "groq"
},
tool_calls: [],
invalid_tool_calls: [],
usage_metadata: {
  "input_tokens": 91,
  "output_tokens": 776,
  "total_tokens": 867
}
}
```



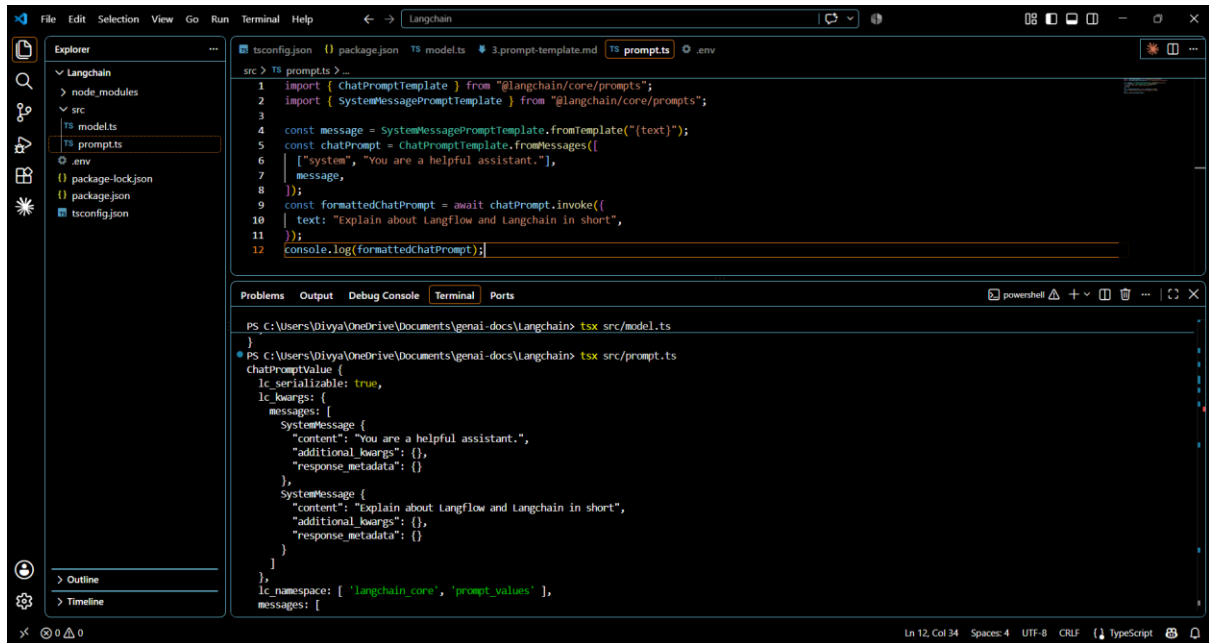
The screenshot shows the VS Code interface with the Explorer sidebar on the left displaying the project structure: `Langchain`, `node_modules`, `src`, `ts models.ts`, `.env`, `package-lock.json`, `package.json`, and `tsconfig.json`. The main editor area shows the `ts models.ts` file with the following content:

```
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/models.ts

{"object": "chat.completion",
  "created": 1786987983,
  "model": "openai/gpt-oss-120b",
  "usage": {
    "queue_time": 0.304761534,
    "prompt_tokens": 91,
    "prompt_time": 0.004791124,
    "completion_tokens": 776,
    "completion_time": 1.632367671,
    "total_tokens": 867,
    "total_time": 1.637158795,
    "completion_tokens_details": {
      "reasoning_tokens": 46
    }
  },
  "usageBreakdown": null,
  "system_fingerprint": "fp_4708260202",
  "x_groq": {
    "id": "req_01m08chah7eaa9srgah8fzq912",
    "seed": 1403737283
  },
  "serviceTier": "on_demand",
  "model_provider": "groq"
},
tool_calls: [],
invalid_tool_calls: [],
usage_metadata: {
  "input_tokens": 91,
  "output_tokens": 776,
  "total_tokens": 867
}
}

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>
```

Prompt.ts

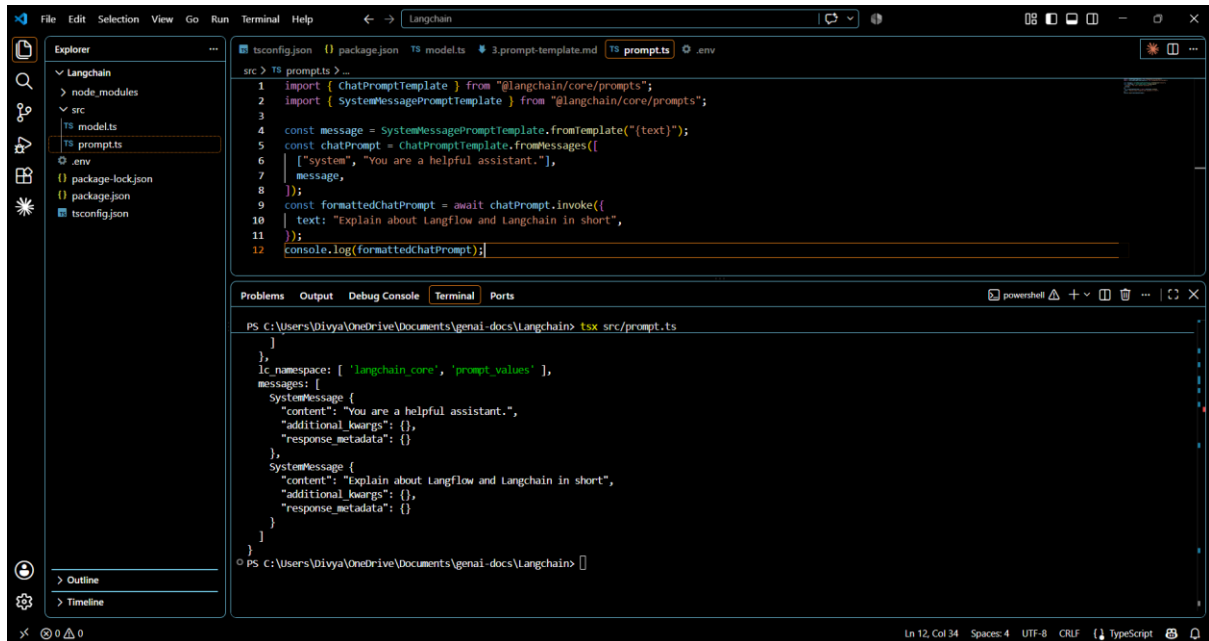


The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure: `Langchain` (root), `node_modules`, `src`, `ts models`, `ts prompts` (selected), `.env`, `package-lock.json`, `package.json`, and `tsconfig.json`. The editor window shows the `prompt.ts` file with the following code:

```
1 import { ChatPromptTemplate } from "@langchain/core/prompts";
2 import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6   ["system", "You are a helpful assistant."],
7   message,
8 ]);
9 const formattedChatPrompt = await chatPrompt.invoke({
10   text: "Explain about Langflow and Langchain in short",
11 });
12 console.log(formattedChatPrompt);
```

The terminal output shows the command `tsx src/model.ts` being executed, followed by the JSON output of the `chatPromptValue` object:

```
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/model.ts
}
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/prompt.ts
chatPromptValue {
  lc_serializable: true,
  lc_kwargs: {
    messages: [
      SystemMessage {
        content: "You are a helpful assistant.",
        additional_kwargs: {},
        response_metadata: {}
      },
      SystemMessage {
        content: "Explain about Langflow and Langchain in short",
        additional_kwargs: {},
        response_metadata: {}
      }
    ],
    lc_namespace: [ 'langchain_core', 'prompt_values' ],
    messages: [
```



The screenshot shows the VS Code editor with the file explorer on the left displaying the project structure: `Langchain` (root), `node_modules`, `src`, `ts models`, `ts prompts` (selected), `.env`, `package-lock.json`, `package.json`, and `tsconfig.json`. The editor window shows the `prompt.ts` file with the following code:

```
1 import { ChatPromptTemplate } from "@langchain/core/prompts";
2 import { SystemMessagePromptTemplate } from "@langchain/core/prompts";
3
4 const message = SystemMessagePromptTemplate.fromTemplate("{text}");
5 const chatPrompt = ChatPromptTemplate.fromMessages([
6   ["system", "You are a helpful assistant."],
7   message,
8 ]);
9 const formattedChatPrompt = await chatPrompt.invoke({
10   text: "Explain about Langflow and Langchain in short",
11 });
12 console.log(formattedChatPrompt);
```

The terminal output shows the command `tsx src/prompt.ts` being executed, followed by the JSON output of the `chatPromptValue` object:

```
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/prompt.ts
}
},
lc_namespace: [ 'langchain_core', 'prompt_values' ],
messages: [
  SystemMessage {
    content: "You are a helpful assistant.",
    additional_kwargs: {},
    response_metadata: {}
  },
  SystemMessage {
    content: "Explain about Langflow and Langchain in short",
    additional_kwargs: {},
    response_metadata: {}
  }
]
}
PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain>
```

Toolintegration.ts

The screenshot shows the VS Code editor with the file `toolintegration.ts` open. The file contains the following code:

```

1 import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";
2 import { MistralAIEmbeddings } from "@langchain/mistralai";
3 import { MongoClient } from "mongodb";
4 import "dotenv/config";
5
6 const client = new MongoClient(process.env.MONGODB_ATLAS_URI!);
7 const collection = client
8   .db(process.env.MONGODB_ATLAS_DB_NAME!)
9   .collection(process.env.MONGODB_ATLAS_COLLECTION_NAME!);
10
11 const embeddings = new MistralAIEmbeddings({
12   model: "mistral-embed",
13 });
14
15 const vectorStore = new MongoDBAtlasVectorSearch(embeddings, {
16   collection,
17   indexName: "vector_index", // Defaults to "default"
18   textKey: "text", // Defaults to "text"
19   embeddingKey: "embedding", // Defaults to "embedding"
20 });
21
22 const similaritySearchResults = await vectorStore.similaritySearch("Registration testcases", 5);
23
24 for (const doc of similaritySearchResults) {
25   console.log(doc);
26 }

```

The terminal output shows the result of running `tsx src/toolintegration.ts`:

```

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/toolintegration.ts
Document {
  pageContent: '',
  metadata: {
    _id: new ObjectId('6a80be27be4983c83083bd94'),
    module: 'Registration',
    id: 'TC_045',
  },
}

```

The screenshot shows the VS Code editor with the file `toolintegration.ts` open. The file contains the following code:

```

1 import { MongoDBAtlasVectorSearch } from "@langchain/mongodb";
2 import { MistralAIEmbeddings } from "@langchain/mistralai";

```

The terminal output shows the result of running `tsx src/toolintegration.ts`:

```

PS C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain> tsx src/toolintegration.ts
Document {
  pageContent: '',
  metadata: {
    _id: new ObjectId('6a80be27be4983c83083bd94'),
    module: 'Registration',
    id: 'TC_045',
    prerequisites: 'Valid from and to Date',
    title: 'Registration Count',
    description: 'Verify Registration count report',
    steps: '1.Launch the application\n' +
      '2.Login to the application with valid credentials.\n' +
      '3.Click on Patient Service Tab and select Registration from the drop down.\n' +
      '4.Navigate to Functions->Reports->Registration count\n' +
      '5.Provide the input details like from and to Date and click on report and observe the report\n' +
      '6.Click on report to Excel button (if required)',
    expectedResults: 'Registration count report should get generate.',
    automationManual: 'Automated',
    priority: 'P1',
    createdBy: 'Prabakaran',
    createdAt: 45186,
    lastModifiedDate: 45279,
    linkedStories: 'MC-016',
    risk: 'Low',
    version: 'v6.0',
    type: 'Integration',
    createdAt: 2026-08-15T19:29:43.252Z,
    embeddingMetadata: {
      model: 'mistral-embed',
      cost: 0.000017056000000000004,
      tokens: 171,
      apiSource: 'mistral-batch',
      batchNumber: 107,
    },
  },
}

```

Chain.ts

The image shows a Visual Studio Code editor window with a dark theme. The Explorer sidebar on the left shows a project structure with files like `tschain.ts`, `tsmodels.ts`, `tsprompts.ts`, `tsintegration.ts`, `.env`, `package-lock.json`, and `tsconfig.json`. The main editor area displays the content of `tschain.ts`, which is a TypeScript file that uses the Langchain library to interact with a ChatGPT model. The code imports `ChatGrok`, `ChatPromptTemplate`, and `StringOutputParser` from `@langchain/grok`, `@langchain/core/prompts`, and `@langchain/core/output_parsers` respectively. It also imports `dotenv/config`. The code defines a prompt template, initializes a `ChatGrok` model with an API key, creates an `StringOutputParser`, and then uses a `chain` to process a prompt. The chain is invoked with a text input, and the result is logged to the console.

```
1 import { ChatGrok } from "@langchain/grok";
2 import { ChatPromptTemplate } from "@langchain/core/prompts";
3 import { StringOutputParser } from "@langchain/core/output_parsers";
4 import "dotenv/config";
5
6 const prompt = ChatPromptTemplate.fromMessages([
7   ["system", "Please review the user query below and determine if it contains any form of toxic behavior, such as insults, threats, or highly negative"],
8   ["user", "{text}"]
9 ]);
10
11 const chatModel = new ChatGrok({
12   apiKey: process.env.GROQ_API_KEY!, // Default value.
13   model: process.env.GROQ_MODEL!,
14 });
15
16 const outputParser = new StringOutputParser();
17 const chain = prompt.pipe(chatModel).pipe(outputParser);
18 const res = await chain.invoke({ text: "You are a complete idiot and your ideas are worthless." });
19 console.log(res);
```

The terminal at the bottom shows the command `tsx src/chain.ts` being executed. The output is a single line: `Toxic`. The terminal also shows the current directory as `C:\Users\Divya\OneDrive\Documents\genai-docs\Langchain`.